

Multi-Viewpoint Anomaly Detection — Project Specification

Project	Observer-driven multi-viewpoint anomaly detection (Phase 2, Masters research)
Institution	Flinders University — STEM9003
Version	1.0 — 8 August 2026
Status	MVP operational (3 LLM backends, live runs verified)
Codebase	<code>poc_california_housing/</code> — Python, plain functions, ~5 files
Audience	Human contributors <i>and</i> AI coding agents. Read §3 before touching anything.

1. Executive summary

Anomaly detection systems answer the question "*is this entity unusual?*" — but the answer depends entirely on **what you look at**. A census block on Santa Catalina Island is the single most anomalous location in California when a detector sees only coordinates, and utterly ordinary when it sees income and population. Neither verdict is wrong; they belong to different **viewpoints**.

This project builds and studies a system in which an **AI agent, given a one-sentence goal, decides the viewpoint itself** — which columns to observe and which rows to compare against — while deterministic statistical code computes every anomaly score. The current deliverable is a deliberately small MVP (one agent, one goal, three interchangeable LLM backends) that already demonstrates the core phenomena: goal-dependent viewpoints, run-to-run derivation variance, and agent self-correction. The research programme grows this into multi-observer experiments where *disagreement between viewpoints* becomes the detection signal.

The one rule that defines the project: the language model never computes an anomaly score, and the statistics never choose what to look at. Break that split and the research is circular.

2. Problem and motivation

Classical anomaly detection fixes the feature space in advance and treats "anomalous" as a property of a data point. In reality, anomaly status is **relative to a purpose**:

- A *census-quality auditor* asks: could this record describe a real place?
- A *regional planner* asks: does this neighbourhood fit its region?

Same data, different questions, different — and equally valid — anomalies. Measured on the California housing dataset (20,640 census block groups, 8 attributes), the effect is not subtle:

- Catalina Island block: **rank 1 of 20,640** under a location-only viewpoint, **rank 287** when all 8 attributes are fused. Fusion diluted the signal.
- Block 19006 (1,243 people per "household" — an institution): **rank 1** under a `[AveRooms, AveOccup]` viewpoint, **rank 4,259** under `[AveRooms, AveBedrms]`. The viewpoint decides whether the most impossible record in the dataset is visible at all.

- In a 3-viewpoint pilot, **89% of flagged entities were flagged by exactly one viewpoint**. Disagreement is the normal case, not the exception.

Existing literature fuses per-view signals into one score (survey: Belay et al., *Sensors* 26(8):2330, 2026 — the field's only agentic-AD survey, in which the words "viewpoint" and "multi-view" never appear). Nothing published derives the analytic frame *from a stated intent*. That derivation step is this project's territory.

3. Core concepts

These four terms are the project's vocabulary. Use them precisely.

3.1 Observer point

A declared **intent** — goal, purpose, rules — stated in natural language by a human.

┃ "Audit this census extract for records that cannot describe a real place."

An observer point contains **no column names, no thresholds, no methods**. It is the input to derivation, never the output.

3.2 Perspective

What "**normal**" means under that intent — the semantic lens the agent forms while exploring. For the auditor intent above: "*normal means the household ratios are mutually consistent.*" For a planner intent: "*normal means typical for its region, not for the whole state.*" The perspective explains the viewpoint; it is recorded in every spec (**why**) so results stay interpretable.

3.3 Viewpoint

The **executable slice of data** the perspective implies:

- **columns** — what is measured (e.g. [AveRooms, AveBedrms, AveOccup])
- **row filter** — who the entity is compared against (e.g. only southern California)

Columns and rows are equally important. Columns decide *what* is observed; the row filter decides the **reference population** — and the same entity can be normal against one population and extreme against another. A viewpoint is written down as a **spec** (a plain JSON dict), which is machine-executable and replayable forever without any LLM.

Derivation chain: observer point → perspective → viewpoint → statistical scores

3.4 The two-plane rule (anti-circularity)

- **Semantic plane (LLM):** reads the intent, explores via tools, emits the spec, explains results. *Never produces a number that acts as a score.*
- **Statistical plane (code):** slices, standardises, runs Local Outlier Factor, produces every score. *Never chooses what to look at.*

Why it matters: an agent that both frames the question and marks the answers is grading its own work — any "finding" would be unfalsifiable. The split is what makes agent-derived viewpoints a research instrument instead of a demo.

3.5 Research context: the anomaly taxonomy (A1–A4)

The wider thesis studies four anomaly classes; the MVP currently exercises A1 only.

CLASS	DEFINITION	EXAMPLE
A1	Extreme within one viewpoint	1,243 people per household
A2	Every viewpoint normal; the <i>combination</i> impossible	Beverly Hills location + farmworker income profile
A3	Flagged by one viewpoint, <i>excused</i> by another's evidence	High occupancy — but it's a prison
A4	A viewpoint itself is stale or unreliable	One data source out of date

Key prior result (pilot): per-viewpoint scores contain essentially **zero A2 signal** — swap-constructed A2 anomalies scored at chance (AUC $\approx$ 0.53) for any combination of per-view scores. Cross-view evidence, not verdict fusion, is the only route to A2. This shapes the roadmap (§8).

4. Scope

Goals

1. **G1 — Derivation:** an agent turns a one-sentence observer point into a valid, executable viewpoint spec, using tools, without seeing raw data rows. *(done)*
2. **G2 — Interchangeable minds:** the same loop runs on a scripted dummy, Claude, or Gemini, so architecture effects and model effects can be separated. *(done)*
3. **G3 — Evidence trail:** every run persists its spec *and* full reasoning trace for later analysis; failed runs included. *(done)*
4. **G4 — Measurement:** compare derived viewpoints against hand-built and random baselines; measure derivation variance across repeats and models. *(next)*
5. **G5 — Multi-observer:** two agents, two goals, disagreement as signal — the research experiments (cells ②, ②½, ③). *(planned)*

Non-goals

- **Not** claiming detection-performance superiority over classical methods.
- **Not** a production anomaly-detection system; datasets are research vehicles.
- **Not** dependent on any single LLM vendor — the backend contract (§6.3) is the only integration surface.

5. Requirements

Functional

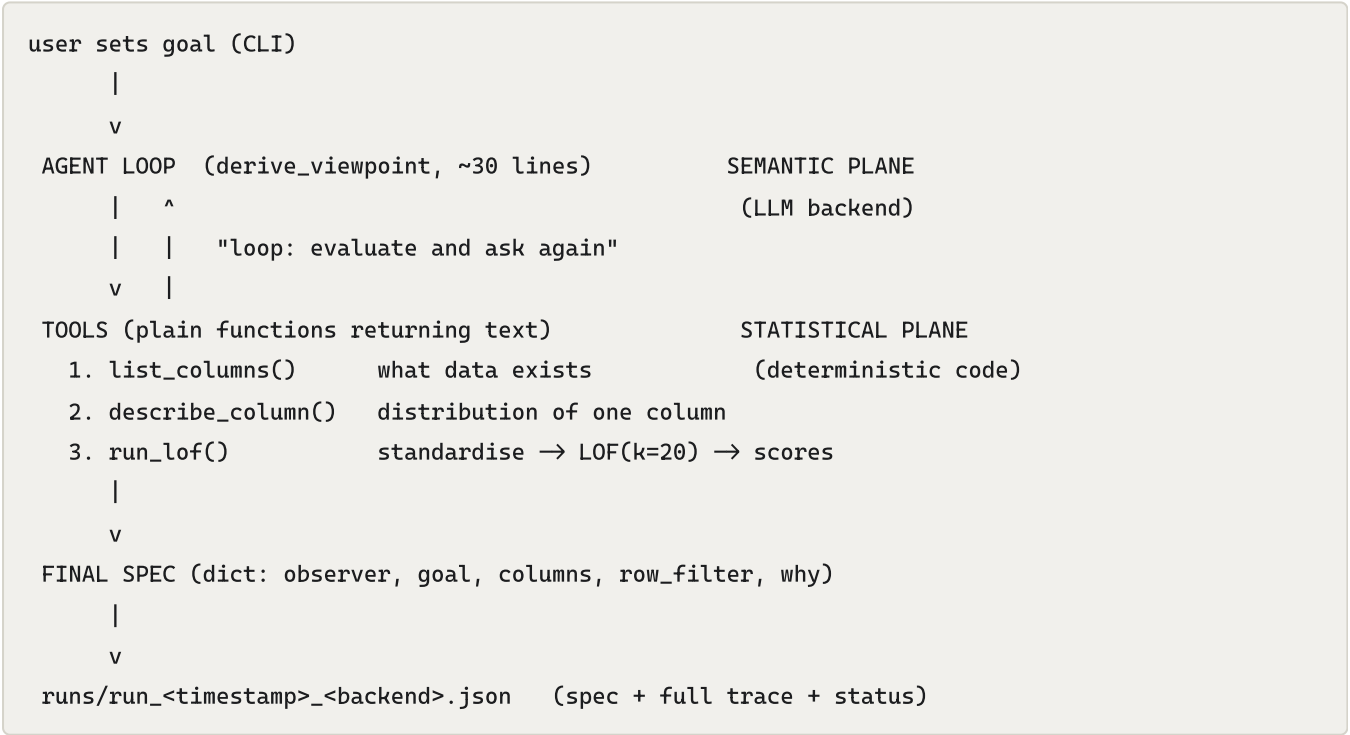
ID	REQUIREMENT	STATUS
FR-1	Accept a goal as a command-line argument; sensible default otherwise	DONE
FR-2	Agent gathers information only through registered tools	DONE
FR-3	Tools return text; errors returned as <code>ERROR: ...</code> strings so the agent can read and self-correct	DONE
FR-4	All anomaly scores produced by <code>run_lof</code> (standardise → LOF k=20); no other scoring path exists	DONE
FR-5	Agent loop hard-capped by <code>max_steps</code> ; prompt budget stated to the model separately	DONE
FR-6	Every run saved to <code>runs/</code> as JSON: spec + full untruncated trace + status (<code>completed</code> / <code>exhausted</code>)	DONE
FR-7	≥3 interchangeable backends behind one contract: dummy (offline), Claude (subscription CLI), Gemini (free API)	DONE
FR-8	Dummy backend deterministic and network-free — the permanent regression test	DONE
FR-9	Any saved spec replayable with zero LLM calls	DONE
FR-10	Null-model harness: derived spec vs hand-built spec vs random column set, same metrics	PLANNED
FR-11	Variance harness: N repeats per goal per backend; spec agreement + steps summarised from <code>runs/</code>	PLANNED
FR-12	Research guards restored on graduation: FIT/DEV/EVAL splits, aggregate-only diagnostics, spec freezing with content hashes	PLANNED
FR-13	Multi-observer cells: one agent two goals; two agents same goal (redundancy control); two agents two goals	PLANNED

Non-functional

ID	REQUIREMENT
NFR-1	Junior-readable: plain functions, no classes; every guard commented with <i>why</i>
NFR-2	Dependencies limited to numpy, pandas, scikit-learn, requests. No LLM SDKs — backends use CLI/REST directly
NFR-3	Secrets never in code or git; keys via environment or gitignored <code>.env</code>
NFR-4	Free-tier friendly: one run $\leq$ ~10 LLM calls; rate limits absorbed by retry; quota failure degrades to a saved partial trace, never a lost run
NFR-5	Windows-first: full-path subprocess calls, no shell-quoting dependence
NFR-6	Run-file schema is append-only: new fields may be added, existing fields never renamed or repurposed

6. Architecture

6.1 System flow



6.2 File map

FILE	RESPONSIBILITY
observer_mvp.py	Dataset, the 3 tools, the agent loop, run persistence, CLI
build_system_prompt.py	The system prompt + prompt builder shared by all real backends
llm_dummy.py	Backend 1 — scripted. The contract is documented here; read it first
llm_claude.py	Backend 2 — Claude via the Claude Code CLI (cclaude -p), no API key
llm_gemini.py	Backend 3 — Gemini REST API (gemini-3.5-flash-lite , pinned), free tier
runs/	One JSON per run, never overwritten
.env	GEMINI_KEY= ... — gitignored

6.3 The backend contract (the only integration surface)

A backend is one function: llm(messages) → dict , returning exactly one of:

```
{ "thinking": "...", "tool": "<name>", "args": { ... } }    # "run this tool for me"
{ "thinking": "...", "final_spec": { ... } }              # "done -- here is my viewpoint"
```

No provider tool-use API is used anywhere: the loop owns the tools; the model names the one it wants, in JSON text. Adding a backend = one new llm_<name>.py file with one function, plus one entry in the BACKENDS dict.

6.4 Guard rails and why each exists

GUARD	WHY
Errors as text, not exceptions	The model reads the error and corrects itself (observed live: wrong column name → fixed next step)
max_steps ≥ prompt budget + 2	The finalising reply consumes a step; an error retry costs another. The prompt number is a request; max_steps is the wall
Standardisation inside run_lof	Without it, the largest-valued column (Population, up to 35,682) decides every distance and every viewpoint gives the same answer
Row filter minimum (100 rows)	In a tiny population everything looks unusual — a filter could manufacture anomalies
Crash/exhaustion still saves the trace	A failed run is evidence, not garbage
Typo-proof CLI	-cclaude (one dash) errors loudly instead of silently running the dummy

6.5 A deliberate MVP relaxation — flagged for graduation

run_lof shows the agent the top-5 flagged rows. Excellent for learning (you watch the agent react to findings); forbidden in the research version, because an agent that sees which entities scored high can tune its viewpoint toward them. FR-12 restores the lock (aggregate-only diagnostics), alongside data splits and spec freezing.

7. Current status — verified results

Five live runs to date (all in `runs/`, each with full trace):

RUN	BACKEND	GOAL	DERIVED COLUMNS	NOTE
195548	Claude	default (impossible places)	AveRooms, AveOccup	catches the prison block, misses Tahoe
202000	Claude	default	AveRooms, AveBedrms	compared 2 candidate viewpoints unprompted; catches Tahoe, misses the prison block (rank 4,259)
203857	Gemini	default	AveRooms, AveBedrms, AveOccup	examined all ratios; catches both
205603	Gemini	regional misfit	— (exhausted)	killed by 20-req/day quota on aliased model; trace saved
205659	Gemini	regional misfit	MedInc, Latitude, Longitude	novel strategy: put coordinates <i>into</i> the LOF space so "region" = the k-nearest spatial neighbours — a third operationalisation of context nobody scripted

Findings worth carrying forward

1. **Derivation variance is real and interpretable.** Same goal, same prompt → different runs choose different, individually defensible viewpoints — and each viewpoint misses something another one catches.
2. **Termination is the agent's own decision** (all completed runs finalised well under budget) — but it stops when its evidence *looks* convincing, and nothing yet asks "what does your viewpoint miss?" Stopping criteria are part of the observer.
3. **Agent self-correction works through error text alone** — no exception handling in the model's path.
4. **Operational lessons:** model aliases resolve unpredictably (pin models for experiments); free-tier quotas are per-model and per-day; rate-limit 429s are normal mid-run events to absorb, not exceptional failures.
5. Pilot baselines from the earlier notebook (max-of-views AUC 0.528 vs joint-LOF 0.756 on swap anomalies) motivated the design but their code was removed in the simplification — treat as indicative until re-established under FR-12.

8. Roadmap

PHASE	CONTENT	EXIT CONDITION
P1. Baselines (next)	FR-10: derived vs hand-built vs random viewpoints, same metrics. FR-11: $N=5 \text{ repeats} \times \{\text{Claude, Gemini}\} \times \geq 2 \text{ goals}$ from <code>runs/</code>	If derived $\approx$ random on every goal, the derivation claim dies here — better in week one than month eight
P2. Harden	FR-12: splits, aggregate-only diagnostics, spec freezing/hashing; pin models; anonymised-column condition (the dataset is in every LLM's training corpus)	MVP relaxations closed; results reproducible end-to-end
P3. Multi-observer	FR-13: cells ② (one agent, two goals), ②½ (two agents, one goal — redundancy control), ③ (two agents, two goals). Contradiction between observers becomes the object of study	Measured: does two-goals-one-mind contaminate? Do complementary observers beat redundant ones?
P4. Cross-view (A2/A3)	Swap-injected A2 ground truth; cross-view evidence exchange; explain-away (A3) adjudication — demote-only, applied by code	The thesis experiments

9. Contributing

9.1 For humans

- **Read order:** this document → `README.md` → `llm_dummy.py` (the contract) → `observer_mvp.py` top to bottom.
- **Style:** plain functions; comments explain *why*, not *what*; every guard rail gets a sentence justifying it. If a junior developer can't follow the file, it doesn't merge.
- **Adding a tool:** one function in `observer_mvp.py` returning a string; register it in `TOOLS`; document it in the system prompt. Ask first whether the tool leaks per-entity information — that decision outlives the MVP.
- **Adding a backend:** copy the shape of `llm_gemini.py`; implement the two-shape contract; add one `BACKENDS` entry; test with the default goal.
- **Before any change is done:** run `python observer_mvp.py (dummy)`. If the scripted run breaks, the *loop* broke. This is the cheapest test in the project.

9.2 For AI agents — invariants (MUST hold after your change)

- **INV-1** Backends return exactly one of the two contract shapes (§6.3). Never add a third shape; extend via new *fields*, not new shapes.
- **INV-2** Tools return `str`. Failures are `"ERROR: ..."` strings to the model — never exceptions across the tool boundary.
- **INV-3** No code path outside `run_lof` produces anomaly scores, and no LLM output is ever used as a score, threshold, or ranking key.
- **INV-4** `max_steps` (in `derive_viewpoint`) $\geq$ the prompt's stated tool budget + 2. If you change either number, change both files (`observer_mvp.py`, `build_system_prompt.py`).

- **INV-5** Every run — completed, exhausted, or crashed — writes a `runs/` file with the full untruncated trace before the process exits.
- **INV-6** `llm_dummy.py` stays deterministic and offline, and `python observer_mvp.py` must complete cleanly before you report done.
- **INV-7** No secrets in code, prompts, logs, or commits. Keys come from the environment or gitignored `.env`.
- **INV-8** Run-file schema: append fields only; never rename or repurpose existing fields (`run_id` , `backend` , `goal` , `status` , `steps_taken` , `final_spec` , `trace`).
- **INV-9** Do not add SDK dependencies for backends; CLI and REST only (NFR-2).

Verification commands

```
python observer_mvp.py                # dummy regression: must finalise, save a run
python observer_mvp.py -claude x      # must exit with the unrecognised-flag error
python observer_mvp.py --gemini "any goal" # live check (needs .env key; free tier)
```

9.3 Things that look like improvements but are regressions

- Returning structured objects from tools "for cleanliness" → breaks INV-2 and the self-correction mechanism.
- Letting the agent see per-entity scores "for better reasoning" → the central validity threat (§6.5); allowed only in the MVP's top-5 display, already flagged.
- Replacing the dummy with a mocked API client → destroys the offline regression path.
- Tuning `k` , thresholds, or preprocessing per viewpoint *by the agent* with free numerics → reopens circularity; keep choices to closed vocabularies when FR-12 lands.

10. Risks and open questions

RISK	STANDING ANSWER
Dataset memorisation — California housing is in every LLM's training data; an agent may "recognise" rather than reason	Anonymised-column condition planned in P2; world knowledge in <i>derivation</i> is acceptable, in <i>detection</i> it is leakage
Prompt fragility — paraphrasing the goal may change the derived viewpoint	Paraphrase-stability test in P1 (5 phrasings × repeats)
Satisficing termination — agents stop at the first convincing story	Candidate-comparison requirement + "state what your viewpoint misses" prompt rule; later, a coded adequacy gate
"Just subspace search" critique — subspace outlier detection (HiCS, SOD) has searched column subsets since 2001	The claim is <i>intent-conditioned</i> derivation, not search; P1's random-baseline comparison measures exactly the difference
Free-tier ceilings	Pinned lite model; per-run cost ≤ ~10 calls; retry + saved-trace degradation

11. Key references

- Chandola, Banerjee & Kumar (2009). *Anomaly Detection: A Survey*. ACM CSUR — the contextual/behavioural attribute split that "viewpoint" operationalises.
- Breunig et al. (2000). *LOF: Identifying Density-Based Local Outliers*. SIGMOD.
- Gao et al. (2011). *HOAD* — origin of the cross-view swap protocol (A2 ground truth).
- Peng et al. (2022). *ALARM*, IEEE TKDE — multi-view attributed-network AD; source of the random-partition challenge the P1 baselines answer.
- Gu et al. (2025). *ARGOS*, arXiv:2501.14170 — LLM-authors-rules, code-executes; the two-plane precedent and the determinism critique (Fig. 4b) that FR-11 answers.
- Belay et al. (2026). *Agentic and LLM-Based Multimodal Anomaly Detection*. Sensors 26(8):2330 — the field survey in which viewpoints do not yet exist.